PC40 – A26

Lab 03: Parallelizing Convolutional Neural Networks (CNN)

J. GABER
gaber@utbm.fr

Important. The subject of this lab is parallel programming. Complete sequential CNN implementations are provided in C++17, Java, and Python. Choose one language, understand the computational structure, preserve its numerical behaviour, and transform that reference into a correct and efficient multithreaded program.

Application: MNIST handwritten-digit recognition.
Main topic: explicit CPU multithreading, decomposition of convolution, shared/private data, synchronization, local gradients, reductions, granularity and performance.
Machine-learning prerequisite: none. The definitions, worked examples and figures needed for this lab are provided below.

Contents

1 Learning outcomes **3**
1.1 What you already know, and what changes here . . . 3
1.2 Organization and suggested progression . . . 3
1.3 Roadmap: what is required and what is optional . . . 4
1.4 How to use this lab . . . 4

2 Preparation and smoke test **4**
2.1 Files supplied . . . 4
2.2 Compile and execute . . . 5
2.3 Dataset and smoke test . . . 5
2.4 Timing scope . . . 6
2.5 Sequential source structure . . . 7

3 Part A – Understanding the CNN and identifying parallelism **8**
3.1 CNN essentials: filters, feature maps, and tensors . . . 8
3.2 A complete forward pass on a small example . . . 9
3.3 The MNIST architecture and its backward dependencies . . . 11
3.4 Data layout, ownership, and dependencies . . . 12
3.5 Sequential loops and the parallel design . . . 13

4 Part B – First explicit multithreaded implementation **15**
4.1 Choose an ownership rule before choosing a threading API . . . 15
4.2 Balanced chunks with no missing or duplicated work . . . 16
4.3 Place synchronization at actual dependencies . . . 17
4.4 Choose useful work per task . . . 18

5 Part C – Parallel training: local gradients and reduction **18**
5.1 Define the algorithm you intend to preserve . . . 18
5.2 Why protecting only the update is insufficient . . . 19
5.3 Identify the reductions inside convolution backward propagation . . . 20

6 Part D – Correctness validation **21**

7 Part E – Granularity and scalability **22**
7.1 Use a controlled experiment 22
7.2 Connect the observations to computation and memory 23
7.3 Keep a minimal implementation log 24

8 Part F – Interpret the performance results **24**

9 Optional extension – Loop ordering and tiling **25**

10 Final synthesis and oral evaluation **25**

A Language-specific guidance **26**
A.1 C++17: capture a different worker index for each thread 26
A.2 Java: one stable index and private scratch space per worker 27
A.3 Python: first identify the running interpreter 28

B Debugging strategy **28**

C Glossary for the laboratory **29**

References and further reading **30**

1 Learning outcomes

1.1 What you already know, and what changes here

In the previous lab, you transformed a sequential fully connected network for MNIST into a multithreaded program. You identified independent work, analysed shared updates, used synchronization, and accumulated local gradients before reducing them. The reference network had 784 inputs, a hidden layer with 64 neurons, and 10 outputs.

Here, the input remains an image throughout the first layers. A small learned filter visits many spatial positions; neighbouring outputs reuse some of the same input values. This gives you several plausible ways to split the same computation. Your objective is to *choose, implement, validate, and compare* these ways.

A CNN is itself a neural network. In this lab, "previous NN" means the previous *fully connected multilayer perceptron* (MLP), while "CNN" means the convolutional architecture specified in this document.

After the lab, you should be able to:

1. compute a small convolution, ReLU, and pooling operation by hand;
2. derive output dimensions and identify the parameters that are learned;
3. distinguish shared reads, private temporaries, and potentially conflicting writes;
4. implement and compare at least two decompositions of the forward computation;
5. explain where synchronization is necessary and where a lock is unnecessary;
6. implement synchronous mini-batch training using local gradients and one update;
7. validate intermediate values and gradients before interpreting timings;
8. explain speedup, slowdowns, memory costs, and the limits of a measured result.

1.2 Organization and suggested progression

The lab follows a single line of reasoning: **understand the sequential computation, identify independent work, build a correct parallel version, validate it, and then measure and explain its efficiency**. You are not given an already-parallelized kernel to tune.

Section 2 establishes the sequential reference and the measurement protocol. **Part A** combines the CNN essentials, a worked forward pass, analysis of the actual MNIST code, and a dependency/ownership audit. **Part B** starts the first multithreaded implementation, with image and row ownership. **Part C** introduces parallel mini-batch training, **Part D** validates numerical correctness, **Part E** studies scalability, and **Part F** interprets the measurements using Amdahl's model. The optional extension and appendices provide further exploration and language-specific help.

Two-session organization. In session 1, run the sequential reference, complete Part A, and implement and validate image ownership (`v1`) in Part B. In session 2, implement the row-owned forward version (`v2`), parallelize mini-batch training (Part C), validate it (Part D), and measure and explain performance (Parts E–F). Consult the worked examples as needed while coding; they are not separate coursework.

The durations are planning targets, not correctness criteria. If row ownership takes longer, preserve and demonstrate the validated image-owned version and explain the missing work at the checkpoint. Keep every validated version. The five short syntheses are collected into the final oral evaluation; no full lab report is required.

1.3 Roadmap: what is required and what is optional

Session	Version / file	Required deliverable	Indicative time
1	sequential reference and design notes	Setup worksheet, hand-calculation exercise, ownership and dependency audit; Synthesis 1 (Part A)	75 min
1	`cnn_parallel_v1`	Image-ownership forward; bit-identical to the reference; Synthesis 2	90 min
2	`cnn_parallel_v2`	Row-ownership forward; bit-identical to the reference	45 min
2	`cnn_parallel_train`	Image-ownership training with local gradients and a fixed-order reduction; Syntheses 3 and 4	75 min
2	`cnn_parallel_final`	Strong-scaling measurements, including the small-batch case $B < T$; Amdahl interpretation; Synthesis 5	60 min

Everything marked **Optional extension** (filter or tile decompositions, dynamic scheduling, the contention experiment, the input gradient dX, loop ordering and tiling) is outside the required work. Attempt it only after the required versions are validated. The oral evaluation uses the grid in Section 10.

1.4 How to use this lab

- **Exercise.** Questions to answer on paper or in your notes: calculations, explanations, diagrams, and reasoning. No code is required unless explicitly stated.
- **Implementation task.** Work that requires writing, modifying, running, measuring, or validating code.
- **Checkpoint:** paragraphs identify what to verify before continuing.
- **Important.** boxes highlight definitions, assumptions and computational contracts.
- Code marked **pseudocode** describes an algorithm; use the supplied source for exact executable signatures.
- Array indices start at zero. Half-open intervals $[a, b)$ include a and exclude b.
- B is the actual mini-batch size and T is the number of CPU workers. The last mini-batch may be shorter.

2 Preparation and smoke test

2.1 Files supplied

The accompanying package contains the following files.

File	Purpose
`PC40_Lab_CNN_Step_by_Step.pdf`	the lab guide (v21).
`cnn_sequential.cpp`	Complete C++17 sequential reference: forward pass, backward pass, mini-batch SGD, checks, and timings.
`CNNSequential.java`	Complete Java sequential reference implementing the same CNN and command-line workflow.
`cnn_sequential.py`	Complete Python sequential reference implementing the same CNN and command-line workflow.
`fetch_mnist.py`	Optional download and decompression of the standard MNIST IDX files.
`README_code.txt` (v7)	Build commands, command-line options.
`benchmark_template.csv`	Measurement table to fill with your own results (one row per run).
`validation_template.csv`	Correctness record (one row per comparison).

The three sequential programs are runnable references for the same CNN training calculation. They use the same architecture, array ordering, numerical formulas, command-line options, deterministic initialization convention, and validation logic. Choose **one language**—C++17, Java, or Python—and keep that language for both your sequential baseline and your multithreaded implementation. Your performance comparison is always made within the language you selected; cross-language timing comparisons are not required.

Important. Programming-language choice. You may complete this lab in **C++17, Java, or Python**. The purpose is to study the parallelization choices, not to learn a particular API. The three supplied sequential implementations represent the same mathematical algorithm. In Python, CPU-bound multithreading must be interpreted in the context of the actual interpreter: a free-threaded CPython build can execute Python threads concurrently, whereas a GIL-enabled build generally cannot execute pure-Python CPU loops in parallel.

Keep each working version of your code before starting the next implementation step.

2.2 Compile and execute

Choose one of the following three baselines. The commands assume a terminal in the directory containing the source files.

Listing 1: C++17 baseline.

```
g++ -O2 -std=c++17 -Wall -Wextra -pedantic cnn_sequential.cpp -o cnn_seq
./cnn_seq --help
./cnn_seq --self-test
./cnn_seq --gradcheck
```

Listing 2: Java baseline.

```
javac CNNSequential.java
java CNNSequential --help
java CNNSequential --self-test
java CNNSequential --gradcheck
```

Listing 3: Python baseline.

```
python3 cnn_sequential.py --help
python3 cnn_sequential.py --self-test
python3 cnn_sequential.py --gradcheck
```

The `--self-test` option runs three fixtures: (1) a 2×2 convolution on a 4×4 input, compared with hand-computed values; (2) `forward()` on one synthetic image, compared with a naive direct implementation of the convolution, pooling and dense layer (the two groupings differ by about 10^{-16}, the threshold is 10^{-12}), and the sum of the probabilities; (3) the pooling tie rule: on a zero image with zero bias every 2×2 window is a four-way tie, and each recorded winner must be the top-left element. The `--gradcheck` option builds a two-filter network, takes one synthetic image, and compares eight analytic gradient entries (K_0, K_7, b_0, b_1, W_0, W_{169}, W_{2000}, c_3; all non-zero) with central finite differences ($\varepsilon = 10^{-6}$, pass threshold 10^{-5}; observed errors are below 3×10^{-10}). Passing these checks confirms the supplied local calculations. It validates neither batch accumulation nor a parallel version that you have not yet implemented.

Create your own parallel source in the same language. For C++, compile with `-pthread`; Java uses `Thread` (or a fixed executor); Python uses `threading.Thread`. The supplied sequential programs have no parallel mode and no `--threads` option. Add an explicit worker-count argument to your own version and record its exact meaning.

2.3 Dataset and smoke test

For MNIST, the reference expects the four uncompressed IDX files in one directory:

Listing 4: Expected filenames inside data/mnist.

```
train-images-idx3-ubyte
train-labels-idx1-ubyte
t10k-images-idx3-ubyte
t10k-labels-idx1-ubyte
```

You can reuse the MNIST files from the previous lab or run:

Listing 5: Obtain MNIST if it is not already available.

```
python3 fetch_mnist.py --output data/mnist
ARGS="--mode train --train 2000 --test 500 --epochs 3 \
      --batch 32 --filters 8 --seed 42 --lr 0.05"
./cnn_seq --data data/mnist $ARGS                 # C++
java CNNSequential --data data/mnist $ARGS        # Java
python3 cnn_sequential.py --data data/mnist $ARGS # Python
```

The download script fetches the four standard MNIST files [2] from public mirrors (the original website is frequently unavailable), checks their MD5 checksums against values written in the script, and decompresses them. The reference loaders then check the IDX magic numbers, the image dimensions, and the image/label counts, and convert pixel intensities to floating-point numbers by dividing by 255. Loading and file decoding are outside the compute timers.

If MNIST is unavailable, start immediately with the offline dataset:

Listing 6: Run without a dataset download.

```
ARGS="--mode train --train 2000 --test 500 --epochs 3 \
      --batch 32 --filters 8 --seed 42 --lr 0.05"
./cnn_seq --synthetic $ARGS                  # C++
java CNNSequential --synthetic $ARGS         # Java
python3 cnn_sequential.py --synthetic $ARGS  # Python
```

Hint. What the synthetic option demonstrates. The generated images represent ten simple patch-position classes with independent noisy training and test examples. They exercise the whole numerical and training pipeline. They are *not handwritten MNIST digits*: a high synthetic accuracy is not a MNIST recognition result. Always identify the dataset in tables and screenshots.

2.4 Timing scope

To exercise the forward pass without training, use:

Listing 7: A fixed-workload forward benchmark.

```
INFER="--synthetic --mode infer --test 500 --filters 8 --seed 42 --repeats 10"
./cnn_seq $INFER --output-csv sequential_infer.csv
java CNNSequential $INFER --output-csv sequential_infer_java.csv
python3 cnn_sequential.py $INFER --output-csv sequential_infer_python.csv
```

The `--batch` option controls training; in inference mode, `--test` specifies the fixed collection of images processed in each repetition. The inference benchmark freezes the initialized parameters, performs an untimed warm-up, and repeats the forward computation on prepared inputs. It does not load a pretrained model. These fixed parameters are sufficient for a compute-kernel benchmark, but their predictions are not evidence of recognition quality. In `train` mode, evaluation uses the parameters learned during that run.

The inference benchmark prints a `checksum`: the sum of the class-0 probabilities over all processed images and repetitions. It consumes the results, and it is a first validation value. With the command above, the three references print `checksum=500.023634460`. Your parallel version must reproduce it. (The C++ `--output-csv` file stores it with 12 significant digits.) For the parallel version, store each

image's class-0 probability in a preallocated array and add the values in image order after the workers have joined, for each repetition. This prevents changes to the summation order. **Timing scope:** the reference implementations include checksum accumulation inside the inference timer. Include the ordered post-join summation within the corresponding timed region of your parallel benchmark, so that the whole-inference measurements are comparable. A separate forward-kernel-only measurement may exclude checksum handling if it is explicitly labelled.

Indicative sequential time (one 2.8 GHz Xeon core)	C++ `-O2`	Java	CPython 3.13 (GIL)
Inference, `--test 500 --repeats 10`	0.45 s	0.59 s	49 s
Training, one epoch of 2000 images	0.26 s	0.28 s	43 s

Important. Policy for the Python track. The Python reference is written in pure Python loops; it does not call NumPy. It is therefore about one hundred times slower than C++ or Java.

- Use a reduced workload for every Python measurement: `--test 100 --repeats 2` for inference (about 2 s; `checksum=19.995653523`) and `--train 320 --test 100 --batch 32 --epochs 1` for training (about 7 s; `train_loss=2.279992`). Keep the same reduced workload for t_1 and t_T.
- With a GIL-enabled interpreter, CPU-bound Python threads do not run in parallel: expect $S_T \leq 1$. Such a result is acceptable *only* if your report demonstrates it (interpreter check of Appendix A) and explains it.
- To observe a real speedup in Python, use a free-threaded build (`python3.13t` or `python3.14t`) and verify at run time that the GIL is disabled.

The correctness requirements of Part D are identical for the three languages.

One invocation aggregates all `--repeats` passes into one measured duration. Repeat the whole invocation at least five times to obtain independent recorded durations for a median and spread; do not confuse internal repeated work with separate timing samples. Record the exact timing scope before making a comparison. The supplied training timer covers the epoch's mini-batch loop: forward and backward computations, gradient accumulation and zeroing, and SGD updates. The samples are processed in their stored order; there is no shuffling, which makes runs deterministic. The timer excludes dataset loading, the allocation of the persistent workspace and gradient buffers, and evaluation. Note that `backward_accumulate()` allocates its temporary arrays (dz, dP, dA) at every call, *inside* the timer; in a parallel version this becomes per-worker allocation traffic that you may decide to move into the private workspace. Your parallel timer must use a matching scope. Include thread creation inside the timed region if your design creates workers during that region; if you use a persistent pool, report its setup cost separately.

2.5 Sequential source structure

Reference operation	What to inspect
`forward()`	Convolution, activation, pooling, flattening, dense layer and probabilities.
`backward_accumulate()`	One image's contributions added to parameter-gradient buffers.
`sgd_update()`	Division by actual batch size and one parameter update.
`train_epoch()`	Mini-batch boundaries, ordering, gradient reset and update frequency.
`evaluate()`	Forward-only evaluation with fixed current parameters.
`inference benchmark`	Warm-up, repeated measurement, and result consumption.

Implementation task. Preparation worksheet. Record your compiler or interpreter/runtime and version, CPU model, physical/logical core counts if known, operating system, dataset, floating-point type, and commands. Identify the model parameters and the per-image workspace in the source. Run the two numerical checks and one short training run. Explain why a smaller loss and a shorter execution time answer two different questions.

Checkpoint: Ready to analyse the computation. You can build and run the sequential program, identify the dataset used, distinguish training from forward-only timing, and locate the complete forward and backward calculations. Keep this reference unchanged for later comparisons.

3 Part A – Understanding the CNN and identifying parallelism

Before introducing threads, inspect one complete sequential CNN computation. The same preparation connects the CNN's layers, tensor layout, numerical operations, data dependencies, and possible work partitions. The objective is *not* a separate machine-learning course: you need enough understanding to explain what each worker will read, compute, and write.

3.1 CNN essentials: filters, feature maps, and tensors

From the previous MLP to a CNN. A **neural network (NN)** is a parameterized composition of operations. Its trainable parameters are adjusted using examples and a loss function. An **MLP**, or multilayer perceptron, usually contains fully connected layers: every output of a dense layer depends on every input to that layer. A **CNN**, or convolutional neural network, includes convolutional layers, in which a small filter is reused at many spatial positions. Both MLPs and CNNs are neural networks. A CNN can contain a fully connected classifier at its end, as ours does.[1]

Important. Three ideas to keep separate. Locality: an output of a convolution depends on a small spatial region of its input. **Parameter sharing**: the same filter coefficients are used at different positions. **Parallelism**: several computations may execute concurrently when their dependencies permit it. Parameter sharing describes the mathematical model; it does not by itself imply a concurrent write or a need for a mutex.

The **hidden features** are intermediate numerical values computed from an image. A **feature map** is the spatial array produced by applying one filter. A filter's weights are trainable parameters; its feature map is an activation that changes with the input. Do not assume in advance that filter 0 must detect vertical lines or that filter 1 must detect curves. Such interpretations must be investigated after training.

From an image to a tensor. A grayscale MNIST image contains 28×28 pixels and one input channel. We divide each stored pixel value by 255, obtaining an input value in $[0, 1]$. For a colour image, the red, green, and blue components would usually form three channels. A **tensor** is simply a multidimensional array. For example, a batch of images can be represented by four dimensions: image, channel, row, and column. In this lab, MNIST images are grayscale, so $C = 1$ and the channel index is always $d = 0$.

In diagrams, image dimensions are written as $H \times W \times C$: height, width, channels. In the implementation, use the **NCHW convention**, with a batch dimension first:

$$X[n, d, i, j], \qquad 0 \le n < B, \quad 0 \le d < C, \quad 0 \le i < H, \quad 0 \le j < W.$$

Here B is the number of images in the current mini-batch, not the number of worker threads. We reserve T for the number of workers. All array indices in this lab start at zero.

Index conventions used throughout the guide. n always denotes an image of the mini-batch; f a convolution filter; (i, j) an input pixel; (r, q) an output position of the convolution or of the pooling; k a class; t a worker. W denotes the dense-layer weights; the image width is written W_s when it appears in a formula.

Recall that a **batch** is a group of input examples processed together. In this lab, one example is one MNIST image. Thus, a batch of size $B = 32$ contains 32 images.

Hint. Logical shape versus memory layout. In the equations, a batch is written as a multidimensional tensor $X[n, d, i, j]$. In the reference programs, each MNIST image is stored more simply as a flat array of 784 values, so pixel (i, j) is accessed at index `i*28+j`. Intermediate CNN values such as S, A, and P are stored in a reusable *one-image* workspace (`WS` / `Workspace`) that has no batch dimension: S is the output of the convolution before activation, $A = \mathrm{ReLU}(S)$ is the activation map, and $P = \mathrm{MaxPool}(A)$ is the pooled activation. When several images are processed in parallel, each worker must use its own workspace.

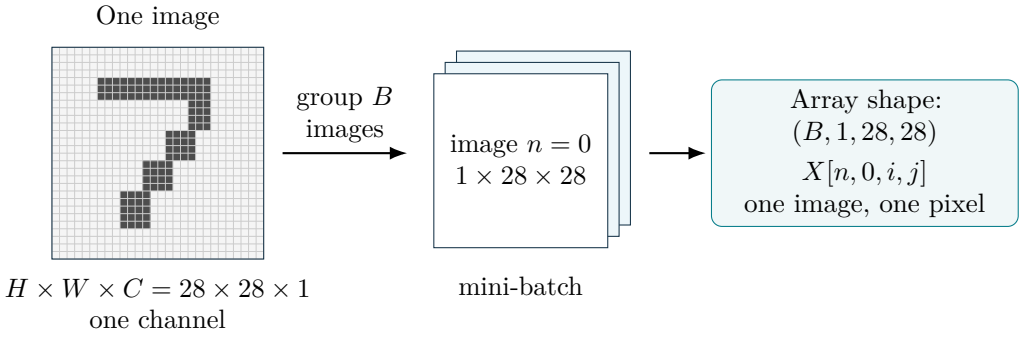


Figure 1: A mini-batch contains several images. For MNIST, each image has one channel. The batch index selects the image; the channel index selects the channel within that image.

Exercise. Identify the data. For a batch of $B = 32$ grayscale images, write the four array dimensions and the number of input values. Then repeat the calculation for 32 colour images of the same height and width. Explain which index would change when you move to the next image, the next channel, and the next pixel in a row.

3.2 A complete forward pass on a small example

Before returning to the full MNIST architecture, it is useful to follow one complete CNN forward pass on very small arrays. The purpose of this example is not to define a useful classifier, but to make every transformation visible:

$$X \longrightarrow S \longrightarrow A \longrightarrow P \longrightarrow h \longrightarrow z \longrightarrow p.$$

Step 1 – Input and convolution. Consider the 5×5 grayscale input

$$X = \begin{bmatrix} 1 & 2 & 0 & 1 & 3 \\ 3 & 1 & 2 & 2 & 0 \\ 0 & 1 & 3 & 1 & 2 \\ 2 & 2 & 1 & 0 & 1 \\ 1 & 0 & 2 & 3 & 1 \end{bmatrix}$$

and one 2×2 filter with zero bias,

$$K = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}.$$

With stride 1 and padding 0, the convolution produces a 4×4 preactivation map. For example,

$$S_{0,0} = 1(1) + 2(0) + 3(-1) + 1(1) = -1,$$

and repeating the same calculation at all valid positions gives

$$S = \begin{bmatrix} -1 & 3 & 0 & -1 \\ 4 & 3 & 0 & 3 \\ 0 & 0 & 2 & 2 \\ 1 & 4 & 2 & -2 \end{bmatrix}.$$

Thus S is the output of the convolution *before* the activation function.

Step 2 – ReLU. Apply $A = \mathrm{ReLU}(S) = \max(0, S)$ element by element:

$$A = \begin{bmatrix} 0 & 3 & 0 & 0 \\ 4 & 3 & 0 & 3 \\ 0 & 0 & 2 & 2 \\ 1 & 4 & 2 & 0 \end{bmatrix}.$$

The negative convolution outputs have become zero; positive values are unchanged.

Step 3 – Max-pooling. Apply 2×2 max-pooling with stride 2. Each non-overlapping 2×2 block of A contributes its largest value:

$$P = \begin{bmatrix} \max\begin{bmatrix} 0 & 3 \\ 4 & 3 \end{bmatrix} & \max\begin{bmatrix} 0 & 0 \\ 0 & 3 \end{bmatrix} \\ \max\begin{bmatrix} 0 & 0 \\ 1 & 4 \end{bmatrix} & \max\begin{bmatrix} 2 & 2 \\ 2 & 0 \end{bmatrix} \end{bmatrix} = \begin{bmatrix} 4 & 3 \\ 4 & 2 \end{bmatrix}.$$

The spatial size has therefore been reduced from 4×4 to 2×2.

Step 4 – Flattening. Read the pooled values row by row to form a one-dimensional vector:

$$P = \begin{bmatrix} 4 & 3 \\ 4 & 2 \end{bmatrix} \quad \longrightarrow \quad h = \begin{bmatrix} 4 & 3 & 4 & 2 \end{bmatrix}^T.$$

Flattening changes the organization of the values, not their numerical values.

Step 5 – Dense layer and logits. For this small example, use three output classes and

$$W = \begin{bmatrix} 1 & 0 & 0 & -1 \\ 0 & 1 & 0 & -1 \\ -1 & 0 & 1 & 0 \end{bmatrix}, \qquad c = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}.$$

The dense layer computes

$$z = Wh + c = \begin{bmatrix} 4 - 2 \\ 3 - 2 \\ -4 + 4 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \\ 0 \end{bmatrix}.$$

The three numbers in z are the **logits**: raw class scores, not yet probabilities.

Step 6 – Softmax. Softmax converts the logits into probabilities:

$$p = \mathrm{softmax}(z) \approx \begin{bmatrix} 0.665241 \\ 0.244728 \\ 0.090031 \end{bmatrix}.$$

The probabilities sum to one, and class 0 has the largest probability.

Exercise. Compute by hand.

1. Keep the same input X and use the filter $K' = \begin{bmatrix} 0 & 1 \\ 1 & -1 \end{bmatrix}$ with zero bias. Compute S', A' and the pooled map P'. Check two of your values by recomputing them from the definition.
2. In the example above, the lower-right block of A is $\begin{bmatrix} 2 & 2 \\ 2 & 0 \end{bmatrix}$. Three positions share the maximum. Which position does the rule "first maximum in row-major order" select? Give its flat index in the 4×4 array A. Explain why sequential and parallel versions must use the same rule, even though P is the same with any rule.
3. For a 28×28 input and a 3×3 filter, give the output size for (a) stride 1, padding 0; (b) stride 1, padding 1; (c) stride 2, padding 0. Use $\lfloor (H + 2p - k)/s \rfloor + 1$.
4. Recover the 13 610 trainable parameters of the MNIST network of Figure 3 from its dimensions.

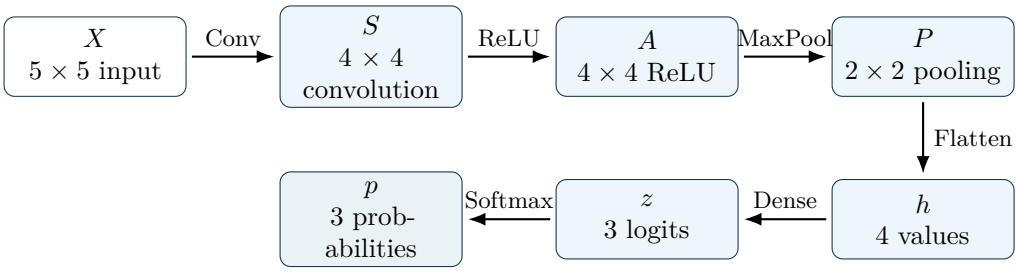


Figure 2: One complete forward pass on a small example. The real MNIST CNN follows the same sequence of transformations, but uses larger arrays, eight learned convolution filters, and ten output classes.

3.3 The MNIST architecture and its backward dependencies

Forward stages and their parallelization implications. For the remainder of the lab, the following compact view is sufficient.

Stage	Operation in this lab	What matters for parallelization
Convolution	3×3, stride 1, padding 0, 8 filters	Different output positions can be computed independently when each output has one owner.
ReLU	$A = \max(0, S)$	Element-wise; no trainable parameters and no shared write is required.
Max-pooling	2×2, stride 2	Each pooled value reads a small region and stores the winning position for backward propagation.
Flatten	$8 \times 13 \times 13 \to 1352$	Only the indexing order changes; the values are not modified.
Dense	$1352 \to 10$ logits	Each logit is a dot product; the weights are shared read-only during a forward pass.
Softmax	10 logits $\to$ 10 probabilities	Requires the ten logits of one image before normalization.

For one MNIST image, the complete forward chain is therefore

$$28 \times 28 \times 1 \to 26 \times 26 \times 8 \to 26 \times 26 \times 8 \to 13 \times 13 \times 8 \to 1352 \to 10 \to 10.$$

The convolution and dense layer contain the trainable parameters. ReLU, pooling, flattening, and softmax do not add trainable parameters.

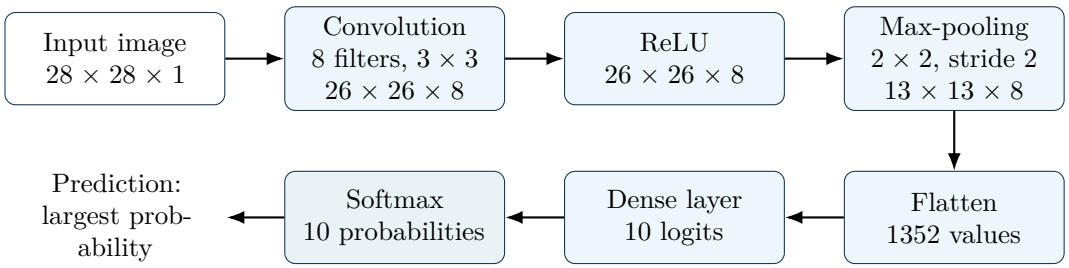


Figure 3: The reference CNN used in the lab. The dimensions shown are for one image.

The reference MNIST architecture. The network has 80 trainable convolution parameters and 13 530 trainable dense-layer parameters, for a total of 13 610. Under the direct-count convention used in this lab, one forward pass contains 48 672 convolution MACs and 13 520 dense-layer MACs. These counts provide useful context for the performance measurements; they do not by themselves predict runtime.

What backward propagation adds for parallelization. The supplied reference code already implements backward propagation. You are not asked to re-derive it. For the parallel version, remember only the following dependencies.

- All images in one synchronous mini-batch must use the same parameter snapshot θ_τ (the parameters after τ updates) during their forward and backward computations.
- Weight sharing means that many output positions contribute to the same convolution-filter gradient. These contributions must therefore be accumulated correctly.
- Each worker can accumulate a private gradient buffer. The private buffers are then reduced before one parameter update.
- Max-pooling backward propagation uses the winning positions saved during the forward pass.
- If an input gradient is computed, overlapping convolution windows can contribute to the same input position; output ownership in the forward pass does not automatically imply disjoint writes in the backward pass.

The learning contract is *synchronous mini-batch SGD*: first accumulate all image gradients at a common parameter snapshot, then reduce and perform exactly one update, dividing by the actual batch size once. Part C gives the formal equations and implementation details.

3.4 Data layout, ownership, and dependencies

The sequential code is the reference for every numerical and memory-access choice. Before making any loop concurrent, identify read-only inputs, locations written by a single worker, and results that require reduction.

Important. Three questions before introducing workers. For every loop, identify (1) the values it reads, (2) the locations it writes, and (3) the calculations that must finish before it starts. Two calculations may run concurrently when their memory accesses and dependencies permit it. A variable being shared does not, by itself, imply that it needs a mutex.

Actual data and flat-array layout. Throughout this part, B is the actual number of images in the current mini-batch, and T is the requested number of CPU workers. The parameters are $\theta = (K, b, W, c)$. The convolution produces preactivations S and activations $A = \max(0, S)$, followed by pooled values P, flattened vectors h, dense logits z, and probabilities. Keep θ unchanged throughout an inference call.

Use the following NCHW convention: batch index, channel index, row, column. For a tensor of shape $[B, C, H, W_s]$ stored in one row-major buffer, where C now denotes the number of channels of *that tensor* (1 for the input image, 8 for S, A and P),

$$\text{offset}(n, f, r, q) = ((nC + f)H + r)W_s + q. \tag{1}$$

Thus columns are contiguous; a whole row is contiguous; a whole feature map is also contiguous. In the C++ and Java source files, the number of output classes is named `N_CLASSES=10` to avoid confusing it with channel count C.

What the reference code actually stores. The supplied programs process one image at a time and keep S, A, P and the pooling winners in a *one-image* workspace, without a batch dimension. They use Equation (1) with $n = 0$:

$$\texttt{si}(f, r, q) = (26f + r)\,26 + q, \qquad \texttt{pi}(f, r, q) = (13f + r)\,13 + q.$$

The flattening index is therefore the pooled offset itself, $j = (13f + r)13 + q$, so that $h_{n,j} = P_{n,f,r,q}$: in the code, h *is* the array `P` and no copy is made. The winner array stores, for each pooled value, the flat `si` offset of the selected element of A.

When a batch dimension becomes necessary. With image ownership, one one-image workspace per worker is enough. With row ownership over the whole batch (Part B), different workers write different rows of the *same* image, and pooling starts only after all rows are complete: you then need batch-wide arrays S, A of shape $[B, 8, 26, 26]$ indexed by Equation (1). For $B = 500$ in double precision, each of them occupies $500 \times 5\,408 \times 8 \approx 21.6\,\text{MB}$. The alternative is to let all workers cooperate on one image at a time with one-image buffers, at the cost of one synchronization point per image and per phase. Choose one and document it.

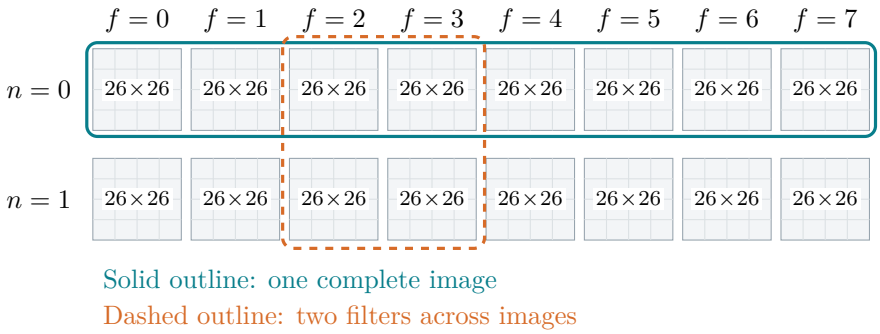


Figure 4: A flat schematic of the output tensor for $B = 2$. Each square is an entire spatial feature map. Image ownership and filter ownership are alternative partitions. Within a square, rows or tiles provide a third partitioning axis.

Implementation task. Audit the sequential implementation. Run a small deterministic case. Record the seed, number of images, batch size, toolchain/runtime settings, initial loss, and accuracy. Locate the loops implementing the convolution, pooling, dense layer, and gradient accumulation. For three selected elements of S and P, calculate the flat offset by hand and compare it with `si` and `pi` in the code. Then give the offset of the same elements in a batch-wide array for image $n = 3$. Finally, draw the dependency chain for one image and mark the arrays that need to survive until backward propagation.

3.5 Sequential loops and the parallel design

The pseudocode below fixes the meaning of the convolution and the ownership of one result. The kernel uses the usual CNN cross-correlation convention: it does not reverse the filter. The full program also handles data loading, stable softmax, loss calculation, and reusable storage.

Listing 8: Sequential forward computation: the reference dependency order.

```
function forward_reference(X, theta):
    for n in 0 .. B-1:
        for f in 0 .. 7:
            for r in 0 .. 25:
                for q in 0 .. 25:
                    acc = b[f]
                    for u in 0 .. 2:         # one filter row at a time
                        acc += ( K[f,0,u,0] * X[n,0,r+u,q]
                               + K[f,0,u,1] * X[n,0,r+u,q+1]
                               + K[f,0,u,2] * X[n,0,r+u,q+2] )
                    S[n,f,r,q] = acc
                    A[n,f,r,q] = max(0, acc)

        for f in 0 .. 7:
            for r in 0 .. 12:
                for q in 0 .. 12:
                    P[n,f,r,q], winner[n,f,r,q] =
                        max_and_first_argmax(A[n,f,2*r:2*r+2,
                                               2*q:2*q+2])

        h[n] = flatten_in_NCHW_order(P[n])   # a view in the code
        for k in 0 .. 9:
            z[n,k] = c[k]
            for j in 0 .. 1351:
                z[n,k] += W[k,j] * h[n,j]
        probabilities[n] = stable_softmax(z[n])
    return saved_forward_values
```

Here slices use a half-open interval: `a:b` includes `a` and excludes `b`. Pooling records the winning input position for backward propagation. The reference implements the tie rule "first maximum in row-major order" with a strict comparison (`if value > best`); keep it. The reference also uses $\mathrm{ReLU}'(0) = 0$ in backward propagation (`S > 0 ?  dA : 0`).

Important. Keep the arithmetic expression, not only the formula. The pseudocode reproduces the grouping used in the code: the three products of one filter row are added together first, then added to `acc`. Writing `acc += K*X` nine times computes the same formula with a different rounding: on the reference data, S then differs by up to 6×10^{-17}. Image, filter and row decompositions do *not* change the operations inside one output value. If you copy the reference inner loop unchanged, your parallel forward pass must therefore reproduce S, A, P, z and the probabilities *bit for bit*. Part D uses this property.

Checkpoint: Synthesis 1/5 – from the sequential CNN to a parallel design. In a few lines, summarize the forward pipeline $X \to S \to A \to P \to h \to z \to p$, its main dimensions, and the trainable parameters. Explain where backward propagation needs reductions. Then state your proposed forward decomposition: which outputs each worker owns, which values are shared read-only, where synchronization is required, and one reason no lock is needed for a shared read. Reuse this synthesis during the final oral evaluation.

4 Part B – First explicit multithreaded implementation

Now that the sequential computation and ownership rules are understood, build a correct multithreaded forward version from your chosen sequential reference, preserving the same numerical result. The key decisions are how to divide the work, what each worker owns, and where synchronization is required.

Start here. Leave the supplied reference untouched. Copy it to `cnn_parallel_v1.cpp`, to `cnn_parallel_v1.py`, or to `CNNParallelV1.java` (rename the class). This copy is version `v1`.

4.1 Choose an ownership rule before choosing a threading API

You will implement two decompositions from Table 1: **image ownership** (`v1`, required) and **row ownership** (`v2`, required). Image ownership is easy to explain and needs no synchronization between layers. Row ownership exposes 208 tasks per image and therefore remains useful when the batch is small, in particular when $B < T$; Part E asks you to measure exactly that case. Filter and tile ownership are optional extensions.

Partition	One task owns	Available work	Question to investigate
Images	Complete forward passes for an image range	B images	What happens when $B < T$?
Filters	Complete maps for selected (n, f) pairs	$8B$ maps	How much input data is read by several tasks?
Rows	Complete rows for selected (n, f, r) triples	$208B$ rows	Are chunks large enough to amortize scheduling?
Tiles	Rectangles within selected maps	Depends on tile size	Which input halos are reread? Is indexing more expensive?

Table 1: Alternative decompositions of the same convolution. The number of logical tasks is different from the number of operating-system threads.

Important. Read-only sharing and exclusive output ownership. During forward propagation, workers may read the same image and the same filter without a mutex. Each output location must have exactly one writer. The accumulator for that location must belong to that writer. All buffers must be allocated before concurrent writes; workers must not resize a shared container. For ordinary numeric array elements, disjoint writes avoid a data race. Adjacent elements may still share a cache line, causing a performance issue called *false sharing*. Avoid packed bit containers for output ownership exercises.

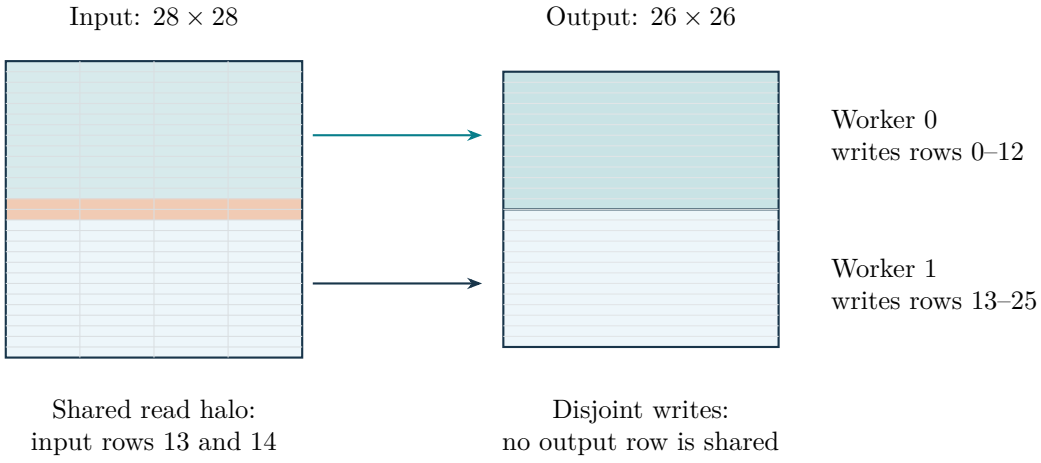


Figure 5: Two row chunks for one image and one filter. Worker 0 reads input rows 0–14; worker 1 reads rows 13–27. Their input regions overlap because the filter has height three. Their output regions remain disjoint.

4.2 Balanced chunks with no missing or duplicated work

Number M work items from 0 to $M-1$. Logical chunk t, where $0 \le t < T$, owns the half-open interval

$$I_t = \left[\left\lfloor \frac{tM}{T} \right\rfloor, \left\lfloor \frac{(t+1)M}{T} \right\rfloor \right). \tag{2}$$

These intervals cover every item exactly once and differ in size by at most one. Some are empty when $T > M$. Use an integer type wide enough for the products; do not introduce an overflow while calculating a boundary.

For row ownership, set $M = B \cdot 8 \cdot 26$. A flat item a identifies $n = \lfloor a/(8 \cdot 26) \rfloor$, $f = \lfloor a/26 \rfloor \bmod 8$, and $r = a \bmod 26$.

Listing 9: Ownership pseudocode for parallel convolution.

```
function conv_forward_owned_rows(X, theta, T, S, A):
    M = B * 8 * 26
    parallel_for t in 0 .. T-1:
        lo = floor(t * M / T)
        hi = floor((t + 1) * M / T)
        for a in lo .. hi-1:
            n = floor(a / (8 * 26))
            f = floor(a / 26) mod 8
            r = a mod 26
            for q in 0 .. 25:
                acc = b[f]              # private to this task
                for u in 0 .. 2:        # same grouping as the reference
                    acc += row_dot(K[f,0,u,:], X[n,0,r+u,q:q+3])
                S[n,f,r,q] = acc         # exactly one writer
                A[n,f,r,q] = max(0, acc)
    # parallel_for returns only after all tasks have completed.
    # Their writes are visible before the next dependent phase.
```

The meaning of `parallel_for` in this handout includes a joining barrier on return. It is an abstract operation, not an existing function supplied by the starter. Implement its semantics using the threading facilities you choose. A persistent pool can execute many logical tasks using a bounded set of workers. With `std::thread`, create the workers first and then join them; joining immediately after each creation would serialize their execution. Thread completion synchronizes with the successful return of `join()`.[3] If you keep a persistent team and separate the phases with a barrier instead of joins, note that `std::barrier` exists only from C++20 on: either compile with `-std=c++20`, or use the small C++17 barrier given in Appendix A. Java offers `CyclicBarrier` and Python `threading.Barrier`.

Exercise. Prove the partition. Calculate the intervals for $M = 10, T = 3$ and for $M = 3, T = 5$. Write down why the intervals cover $[0, M)$ without overlap.

Implementation task. Implement the two forward decompositions. In `v1`, implement image ownership: worker t processes the images of chunk I_t with its own one-image workspace. Add a worker-count option (for example `--threads`). Validate `v1` before continuing: with the inference command of Section 2, the checksum must equal the sequential one exactly, for $T = 1, 2, 3$ and for a T larger than the number of images. Then copy the validated `v1` to `cnn_parallel_v2.*` (`CNNParallelV2.java`) and implement row ownership for the convolution, with the pooling and dense phases starting only after all rows are complete. Keep the arithmetic inside one convolution output unchanged. Record whether workers are recreated at each call or reused.

Optional extension. Static versus dynamic chunks. Equation (2) assigns the same amount of work to every worker. On processors with heterogeneous cores (performance and efficiency cores) or under system load, the slowest worker then determines the elapsed time. Implement a dynamic variant in which workers repeatedly claim the next small chunk from a shared atomic counter (`std::atomic<int>::fetch_add`, `AtomicInteger.getAndAdd`). Compare the two schedules for several chunk sizes. Ownership is still exclusive: each chunk is claimed exactly once.

4.3 Place synchronization at actual dependencies

When an image belongs to one task, that task may execute its convolution, pooling, and dense layer in ordinary program order. It need not wait for other images between its layers. When several workers cooperate on one image, consumers must wait for the data they need. A simple first design completes the whole convolution phase before pooling, then the whole pooling phase before the dense layer. This is conservative but easy to validate.

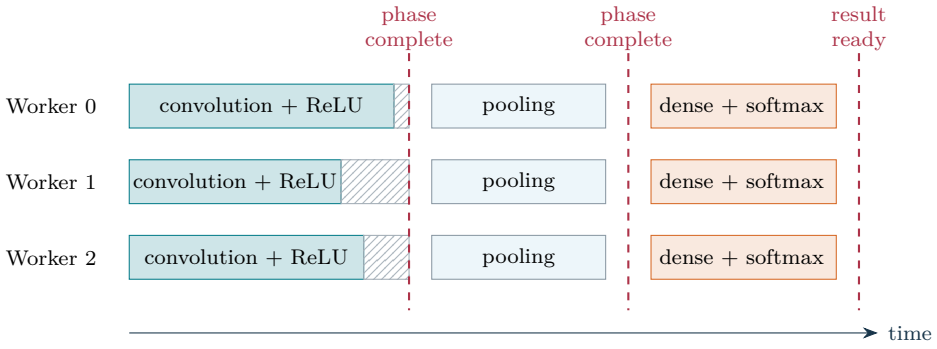


Figure 6: A possible schedule when workers cooperate within images. Hatched regions represent waiting. The vertical boundaries express required visibility of completed data; the illustrated durations are schematic, not measurements.

Phase	Shared, read-only	Output with one owner	Private temporary state
Convolution / ReLU	X, K, b	A subset of S, A	Loop indices, scalar accumulator
Pooling	Completed A	A subset of P, winner indices	Local maximum and its position
Dense/softmax	Completed h, W, c	Logits and probabilities for owned images or outputs	Dot-product sum; softmax maximum and denominator
Batch backward	Fixed θ_τ, input data	One gradient buffer per worker	Saved activations, backward arrays
Reduction / update	Completed local gradients	Reduced gradient; then $\theta_{\tau+1}$	Reduction accumulator

Table 2: An ownership audit. The exact output partition depends on your design. Softmax requires all logits of its own image to be available.

Checkpoint: Explain correctness before timing. Before continuing, identify one input value read by two workers and one output value written by exactly one worker. Explain why the former needs no lock, identify the lifetime of the private accumulator, and point to the operation that makes the completed layer visible to its consumer. Explain how your answer changes between image ownership and row ownership.

4.4 Choose useful work per task

A task is a unit of work submitted to an execution mechanism. An operating-system thread is a resource that executes tasks. Creating one thread for each of the 5 408 convolution outputs of one image would introduce thousands of thread creations for tiny computations. Use chunks and a bounded worker count. Very large chunks can instead leave workers without work, especially near the end of a phase. An intermediate granularity must be chosen by measurement.

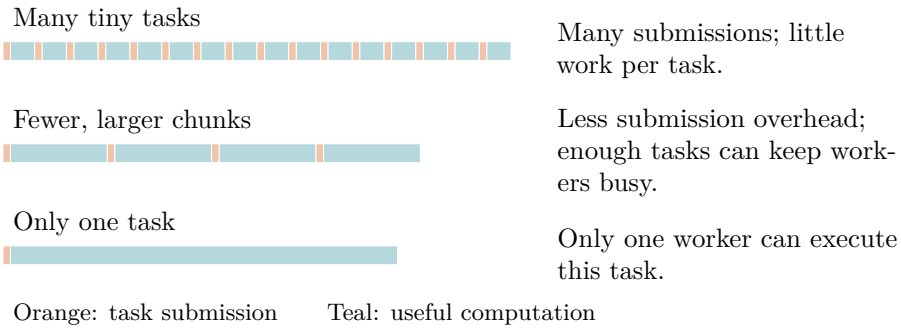


Figure 7: Conceptual task packaging for the same useful work. Ribbon lengths illustrate repeated submission costs only; they are not elapsed times or a prediction of speedup. Cache behavior and load balance also affect the result.

Checkpoint: Synthesis 2/5 – parallel forward. Summarize your first multithreaded forward implementation. State the decomposition used, what each worker owns, the synchronization points, and whether the results match the sequential reference. Record one first timing observation, even if there is no speedup yet. Keep this synthesis for the final summary note.

5 Part C – Parallel training: local gradients and reduction

In this part, you extend the parallel program from forward computation to training. The objective is to preserve synchronous mini-batch SGD by keeping one parameter snapshot during the batch, accumulating gradients locally in each worker, reducing them, and performing a single update.

5.1 Define the algorithm you intend to preserve

All images of mini-batch number τ must use the same parameter snapshot θ_τ. Let $\ell_n(\theta_\tau)$ be the loss for image n and let worker t own image set I_t. Store the *sum* of its per-image gradients:

$$G_t = \sum_{n \in I_t} \nabla_\theta \ell_n(\theta_\tau), \qquad G = \sum_{t=0}^{T-1} G_t, \tag{3}$$

$$\theta_{\tau+1} = \theta_\tau - \frac{\eta}{B} G. \tag{4}$$

Divide exactly once by the actual batch size, including a shorter final batch. Do not average each worker's gradient and then average the workers: when chunks have unequal sizes, that would weight samples incorrectly.

Important. The sequential comparator must implement the same update. Sequential mini-batch training accumulates all B gradients at fixed θ_τ, then performs one update. Per-image SGD updates after every image and usually gives a different trajectory. Compare your parallel program with the sequential mini-batch algorithm using the same initial parameters, ordered images, batch boundaries, learning rate, loss definition, and update count.

Listing 10: One synchronous mini-batch step with complete local gradients.

```
function synchronous_batch_step(batch, theta, eta, T):
```

```
    B = actual_number_of_images(batch)
    # theta is shared and immutable until the update below.
    parallel_for t in 0 .. T-1:
        zero(G_local[t])       # K, b, W, c: the full network
        reset(local_loss[t])
        for n in balanced_image_chunk(t, B, T):
            forward(x[n], theta, workspace[t])
            local_loss[t] += loss(workspace[t], y[n])
            # reference signature: backward_accumulate(x, y, p, w, g)
            backward_accumulate(x[n], y[n], theta,
                                workspace[t], G_local[t])
    # Joining barrier: all forward and backward calls have ended.
    G = fixed_order_sum(G_local[0], ..., G_local[T-1])
    loss = fixed_order_sum(local_loss) / B
    theta = theta - (eta / B) * G    # exactly ONE update
    return loss
```

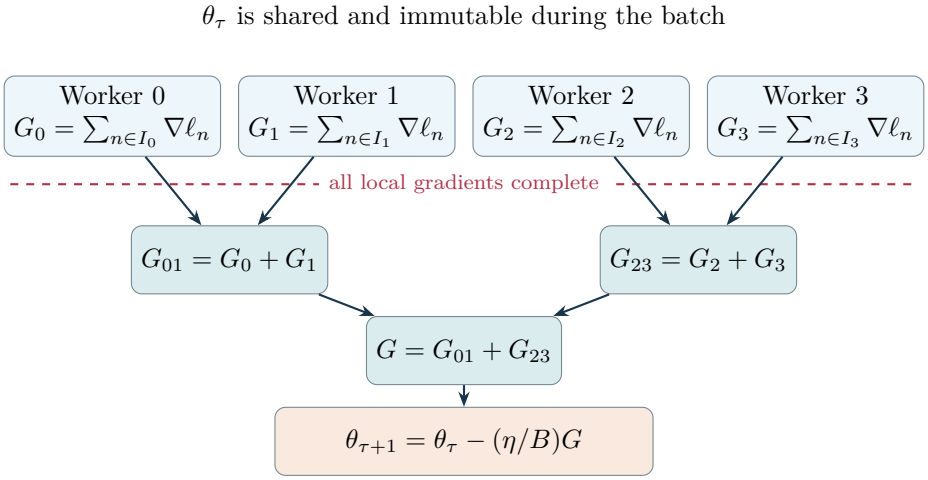


Figure 8: Local accumulation, a fixed reduction tree, and one update for $T = 4$. Every G_t contains gradients for the entire network, including the dense layer. A serial reduction in increasing worker order is also a valid first implementation.

5.2 Why protecting only the update is insufficient

A mutex around `update()` alone does not protect parameter reads in `forward()` and `backward()` against another worker's writes. The exact language-level consequence of a conflicting unsynchronized access depends on the language. In C++, two potentially concurrent accesses to the same memory location, at least one of which is a write and at least one of which is not atomic, form a data race unless one happens before the other; a data race causes undefined behaviour [3]. Java uses its own memory-model rules [5]; Python execution additionally depends on whether the interpreter is GIL-enabled or free-threaded [6]. In all three languages, the algorithmic requirement is the same: workers must not observe parameter updates while computing gradients for one synchronous mini-batch. Even if every access is made race-free, updating between samples changes which parameter version each gradient uses. Race freedom and preservation of the synchronous learning algorithm are separate requirements.

Study lost updates using the deterministic schedule in Figure 9. If you reproduce it, execute the steps sequentially rather than using concurrent code with data races.

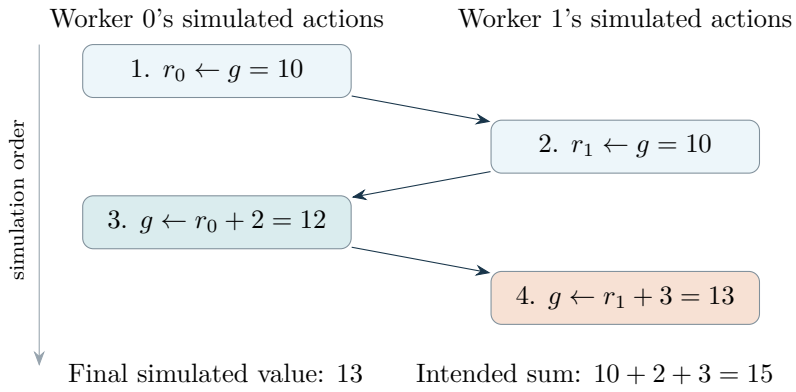


Figure 9: A fully controlled sequential simulation of a lost-update schedule. Both additions use an old value. Local accumulators followed by a reduction avoid this shared read–modify–write pattern during gradient calculation.

Implementation task. Preserve the learning semantics. Copy your validated image-ownership version `v1` to `cnn_parallel_train.*` or `CNNParallelTrain.java`, and extend that copy to parallel training following Listing 10: each worker runs complete forward and backward passes for the images it owns. Implement one local full-network gradient buffer and one reusable workspace per worker. Start with a serial reduction in increasing worker order. Verify that `backward_accumulate()` adds to the supplied buffer and that no worker calls `sgd_update()`. Explain whether a mutex around the complete forward–backward–update sequence would implement this mini-batch algorithm, and whether that design would expose useful parallel work.

Hint. Why training uses image ownership. Row ownership in training would also split the backward pass of one image across workers. Each worker would then hold a partial spatial sum of G_K and G_b for the same image (Equations (5)–(6)), and these partial sums would have to be reduced as well. The dense-layer gradient and dP would need their own partition. This is a valid design, but it is not required here. If you attempt it as an extension, describe the additional reduction explicitly.

Optional extension. Measure the cost of contention. Write two further correct variants of the training step: (a) all workers add into one shared gradient buffer protected by a single mutex, taken once per image; (b) the same, but with one lock per image and per parameter block, or with atomic additions in C++/Java. Both are free of data races and implement the same algorithm (up to rounding). Compare their time with your local-gradient version for $T = 1, 2, 4$ and explain the difference.

5.3 Identify the reductions inside convolution backward propagation

Let $\delta_{n,f,r,q} = \partial \ell_n / \partial S_{n,f,r,q}$ for the per-image loss. For the single-channel convolution,

$$G_K[f, 0, u, v] = \sum_{n=0}^{B-1} \sum_{r=0}^{25} \sum_{q=0}^{25} \delta_{n,f,r,q} \, X_{n,0,r+u,q+v}, \tag{5}$$

$$G_b[f] = \sum_{n=0}^{B-1} \sum_{r=0}^{25} \sum_{q=0}^{25} \delta_{n,f,r,q}. \tag{6}$$

The repeated use of a filter in forward propagation becomes a spatial reduction in backward propagation. Image ownership adds a second reduction across the mini-batch. Each worker performs its spatial sums locally, and the final reduction combines workers. Dense-layer gradients require the same ownership discipline; protecting only the convolution gradients is insufficient.

If you compute the input gradient, different output positions can contribute to the same input position. Parallelizing a *scatter* operation that adds directly into shared dX therefore requires further design. A *gather* alternative gives one worker ownership of each input-gradient value and sums all valid contributors there:

$$dX_{n,0,i,j} = \sum_{f=0}^{7} \sum_{u=0}^{2} \sum_{v=0}^{2} K_{f,0,u,v}\, \delta_{n,f,i-u,j-v}, \tag{7}$$

where out-of-range δ entries are treated as zero. The reference does not compute dX: the raw image is not trainable. Computing it is an optional extension; it becomes necessary when another trainable layer precedes the convolution.

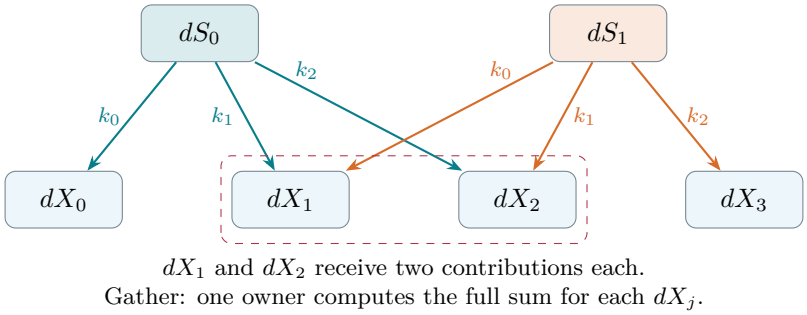


Figure 10: A one-dimensional backward example with a filter of width three and stride one. Overlap between receptive fields creates shared destinations for a scatter implementation. The same issue appears in two dimensions.

Checkpoint: Synthesis 3/5 – parallel training. Summarize how training is parallelized. State why all workers must use the same parameter snapshot during a mini-batch, where gradients are accumulated locally, how the local gradients are reduced, and where the single parameter update occurs. Keep this synthesis for the final summary note.

6 Part D – Correctness validation

Before interpreting any speedup, you must establish that the parallel program still computes the intended algorithm. The synchronous mini-batch update is defined once in Section 5, in particular by Equation (4). This section does not redefine that algorithm: it only verifies numerically that your implementation preserves it.

Use deterministic input, initialization, and batch ordering. Begin with one image, then a non-divisible case such as $B = 5, T = 3$, then more workers than images. Writing a small `dump` routine that saves S, P, z and the four gradient blocks to a text or binary file, in the sequential and in the parallel program, makes these comparisons a one-line script. Include negative preactivations and a controlled pooling tie in the small checks. A correct result should not depend on every chunk having the same size.

Forward values: exact equality. If your inner loop is the reference loop, the forward decompositions of Part B change *who* computes each output, not *how* it is computed. Compare S, A, P, winner indices, z and the probabilities for exact equality, and the inference checksum to all printed digits. A non-zero difference is a bug or a modified expression, not rounding. (In C++, do not change optimization flags between the two builds: a compiler may contract `a*b+c` into a fused multiply–add differently in two code paths; `-ffp-contract=off` removes that possibility.)

Gradients and parameters: tolerance. A reduction over workers changes the order of the additions of G_K, G_b, G_W, G_c and of the loss. Compare these arrays element by element using a combined absolute and relative rule:

$$|a_j - b_j| \le \varepsilon_{\text{abs}} + \varepsilon_{\text{rel}} \max(|a_j|, |b_j|). \tag{8}$$

For double precision, $\varepsilon_{\text{abs}} = 10^{-10}$ and $\varepsilon_{\text{rel}} = 10^{-8}$ can be starting values for small cases, not universal guarantees. Record the maximum observed errors and justify any tolerance change. Check for non-finite values explicitly: a NaN must fail the comparison. Floating-point addition is not associative, so a different valid reduction order may change the last digits.

Comparison	Evidence to retain
Convolution S, ReLU A	Exact equality; first differing index if any.
Pooling P, flatten h	Exact equality of values and winner indices.
Logits, probabilities, checksum	Exact equality; finite values, probability sums.
All gradients G_K, G_b, G_W, G_c	Comparison at the same fixed θ_τ, before division.
One parameter update	Maximum parameter difference after one identical mini-batch.
Several training steps	Loss sequence and update count under fixed data order.

Table 3: Correctness evidence to retain. Accuracy alone cannot establish that the same mathematical computation was performed.

A fixed reduction tree gives a fixed addition order for a fixed partition. Changing T changes the partition and may change rounding. Do not claim bit-for-bit reproducibility across worker counts or machines unless your implementation explicitly establishes it. Once these checks pass, longer training can assess whether learning behavior remains consistent.

Checkpoint: Demonstrate one complete training step. Display the sequential and parallel loss, all four gradient blocks, and the largest parameter difference after one update. State where gradients are zeroed, where the batch-size division occurs, and why every gradient used θ_τ. Demonstrate an uneven image partition. Explain how a good accuracy could conceal a wrong reduction or an update performed too often.

Checkpoint: Synthesis 4/5 – correctness. Summarize the correctness evidence in a few lines. Report the maximum numerical error observed, one non-divisible partition case, and the result of one complete parameter update comparison. Keep these values for the final summary note.

7 Part E – Granularity and scalability

Once correctness has been established, you can study performance. In this part, you measure execution time for different worker counts, compute speedup, efficiency, and throughput, and relate the observed behaviour to granularity, memory use, synchronization, and reduction overhead.

7.1 Use a controlled experiment

Measure two scopes separately: (1) convolution alone, and (2) complete forward inference or a complete training step. Specify whether the second scope includes allocation, worker creation, synchronization, reduction, and update. Keep file reading and interactive printing outside a compute-kernel timing. If they are included in an application timing, identify them explicitly.

Implementation task. Establish a reproducible measurement protocol. After the training version passes the correctness checks, copy it to `cnn_parallel_final.*` or `CNNParallelFinal.java`. Build the sequential and parallel versions with the same language, toolchain/runtime configuration, optimization settings where applicable, numerical precision, and workload. Warm up the computation, then perform at least five independent timed repetitions and report the median and a spread such as the minimum–maximum interval. Reinitialize parameters outside the timer when repeated training measurements must start from identical states. Use a monotonic clock; ensure measured results are consumed so the compiler cannot remove the work. Record CPU model, available hardware threads, operating system, compiler version, and the execution environment's CPU allocation. Laptop processors raise their clock frequency when only one core is busy (turbo); t_1 is then measured at a higher frequency than t_T, and the speedup is underestimated. Note whether your machine has performance and efficiency cores. Use the `benchmark_template.csv` columns for every run.

Choose a bounded worker range, for example $1, 2, 4, \ldots, T_{\max}$ with $T_{\max}$ consistent with the CPU resources available to the process. Check whether a numerical library also creates threads: T application workers each starting another team can oversubscribe the CPU. Record relevant library thread settings. Reuse allocations and workspaces where possible, and keep their placement consistent across comparisons.

For strong scaling, hold the number of images, image size, filter count, batch size, and number of calls or updates fixed while varying T. Define

$$S_T = \frac{t_1}{t_T}, \qquad E_T = \frac{S_T}{T}, \qquad \text{throughput} = \frac{N_{\text{images}}}{t_T}. \tag{9}$$

State whether t_1 is the optimized sequential reference or your one-worker execution mechanism. Prefer reporting both if their overhead differs materially. All compared timings must cover the same work and the same scope.

Quantity to report	What it represents
Workers T	Number of application workers used for the run.
Convolution median time	Median time for the convolution-only timing scope.
Total median and spread	Median and minimum–maximum range for the complete measured scope.
Speedup S_T	t_1/t_T for the same fixed workload and timing scope.
Efficiency E_T	S_T/T.
Throughput	Images processed per second.

Table 4: KPI values to include in your own compact results table for each decomposition and fixed workload.

Varying B or the number of filters is a separate workload experiment. Repeat the strong-scaling study at each selected size. If work grows with T, label the study as weak scaling and define what stays constant per worker. Do not mix its ratios with fixed-work speedups. A modified filter count also changes the dense input size and parameter count; keep the baseline architecture for the required comparisons before attempting that extension.

7.2 Connect the observations to computation and memory

The network has 13 610 trainable scalar parameters. A complete gradient buffer in double precision therefore requires

$$13\,610 \times 8 = 108\,880 \text{ bytes per worker}, \qquad M_{\text{grad}} = 108\,880T \text{ bytes}. \tag{10}$$

This count excludes per-worker activations, winner indices, backward arrays, the reduced gradient, parameters, and input storage. A one-image workspace can be reused as a worker processes its chunk.

A workspace holding the whole batch has a different memory cost. Document your actual allocation strategy.

Count arithmetic and memory separately. In this architecture, convolution performs $B \cdot 8 \cdot 26^2 \cdot 9$ multiply–accumulate terms. Counting each multiply and each addition as a separate operation gives twice this number, before bias handling and activation costs. State your counting convention. Multiple output positions reuse an input value, and multiple filters reuse an image. Row or tile boundaries introduce overlapping input halos. These facts do not imply an equal number of transfers from main memory: caches may serve many repeated reads. A simple traffic estimate must name its reuse assumptions. [1]

At this stage, focus on measured costs: execution time, synchronization, reduction, memory use, and load balance. The theoretical interpretation of the observed speedup is introduced once, in Part F (Section 8).

Implementation task. Measure the case where the batch is too small. With image ownership, at most B workers have work. Measure `v1` and `v2` in inference on a workload with fewer images than workers, for example `--test` 1 with enough repetitions for a run of at least one second, and $T = 1, 2, 4, \ldots$ Report both speedup curves on the same plot. Explain which decomposition can use the additional workers and what it pays for that (number of tasks, synchronization after the convolution phase, batch-wide buffers).

Implementation task. Explain a result, including a disappointing one. Use the preserved `v1`, `v2`, and `final` versions to compare both forward decompositions on at least a small and a larger fixed workload. For each, report correctness evidence, timing distributions, speedup, efficiency, and additional memory. Explain one observed limitation using measured phase times and your ownership design. Consider granularity, insufficient tasks, barriers, gradient reduction, allocation, locality, and oversubscription only when they are supported by your observations. A slowdown accompanied by a correct analysis is a valid experimental result.

7.3 Keep a minimal implementation log

For each preserved version, retain only the information needed to reproduce and compare it: source filename, decomposition, worker mechanism, private buffers, synchronization points, validation status, timing scope, and exact execution command. These notes feed directly into Synthesis 5 and the final oral evaluation; no separate report is required.

Checkpoint: Synthesis 5/5 – performance. Record the main KPI values for each tested worker count: execution time, speedup, efficiency, throughput, and additional memory if measured. Plot execution time and speedup versus the number of workers. In a few lines, explain the observed trend and identify one limiting cost supported by your measurements.

8 Part F – Interpret the performance results

In this part, you interpret the measurements obtained previously rather than reporting speedup alone. You compare the observed behaviour with a simple theoretical model and identify which measured costs prevent ideal scaling.

Suppose a fraction φ of the *measured sequential execution time* is accelerated perfectly by T workers, while the remaining fraction stays sequential. Ignoring synchronization and memory overhead gives Amdahl's model [7]:

$$S_{\text{model}}(T) = \frac{1}{(1-\varphi) + \varphi/T}. \tag{11}$$

The fraction is a fraction of *time*, not a fraction of source-code lines, parameters, or arithmetic operations. Measure it with the same input and implementation that you use for the comparison. A different loop order or memory layout can change it.

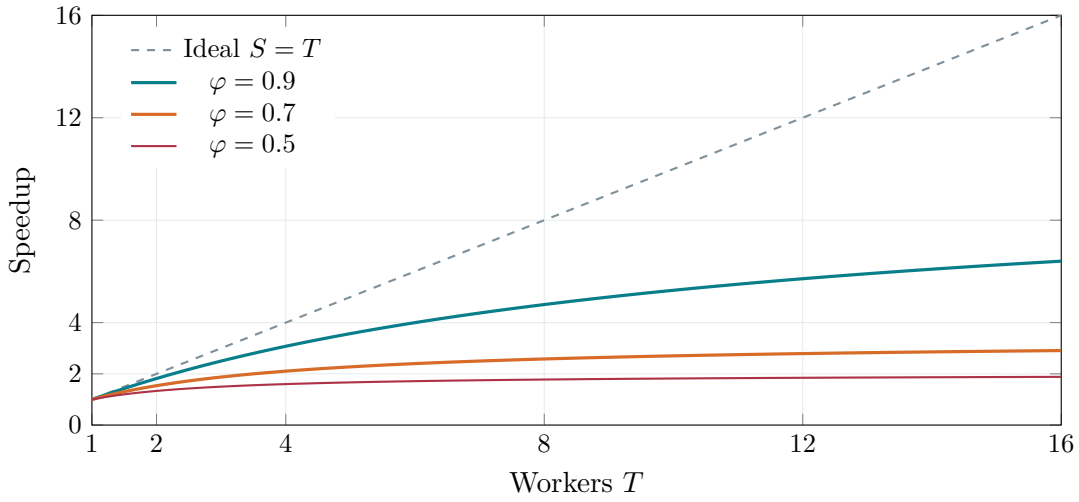


Figure 11: Illustrative curves computed from a model. These are **not measured CNN results**. Even this overhead-free model is limited by the time that remains sequential.

Exercise. Interpret a hypothetical result. If convolution takes 70% of sequential time, calculate the predicted whole-program speedup when only that convolution becomes four times faster. With $\varphi = 0.70$ and $T = 4$, the model gives $1/(0.30 + 0.70/4) \simeq 2.11$. Then explain why a convolution speedup of 4 does not imply a whole-program speedup of 4.

Implementation task. Interpret your measured result. Use measurements from your preserved `final` version. Replace the hypothetical 70% by a measured fraction φ (time the parallelized phases with $T = 1$ and divide by the total time), and discuss scheduling, memory traffic, or other costs that the model omits.

An observed difference from the model is a question to investigate. For example, a parallel implementation may have worse locality, extra barriers, or different allocation behaviour. A result above this simple prediction can also occur if the implementation changes cache use or removes overhead. Report those changes instead of interpreting worker count alone as the cause.

9 Optional extension – Loop ordering and tiling

This optional part explores how implementation choices inside the same mathematical convolution can change performance. You compare loop ordering or spatial tiling while keeping the numerical operation unchanged, and relate any difference to locality, reuse, and temporary storage.

Keep the same operation and compare loop orders or spatial tiling. Measure both numerical agreement and execution time. Convolution can also be expressed through matrix multiplication, but explicitly constructing a patch matrix may introduce data replication and memory traffic. Several convolution algorithms can avoid explicitly storing that matrix [1].

This extension asks you to explain a tradeoff: fewer memory fetches, more contiguous accesses, a larger working set, or extra temporary storage. It does not require implementing every optimized algorithm. Distinguish an *implementation change* from a change in the learned model or the required mathematical result.

10 Final synthesis and oral evaluation

Do not write a full report. Prepare a concise guided synthesis, preferably no more than two pages, assembled from Syntheses 1–5 completed during the lab. It should contain:

- the chosen programming language and machine/compiler configuration;

- the parallel decomposition(s) implemented and the main ownership rule;
- the main correctness evidence, including the maximum numerical error observed;
- a compact KPI table with execution time, speedup, efficiency, throughput, and any relevant memory overhead;
- the requested performance plots, including execution time and speedup versus the number of workers;
- short answers to the key questions raised in the lab: where synchronization is necessary, why some shared reads need no lock, why gradient reduction is required, and what limits the measured speedup;
- a short conclusion explaining what worked, what did not scale as expected, and what you would improve next.

Checkpoint: Oral evaluation. At the end of the lab, call me for the oral evaluation. Be ready to run your final program and use your synthesis note to show the key evidence: correctness, KPI values, performance curves, and the code implementing ownership or gradient reduction. You will also be asked a few short questions about your design choices and their performance consequences.

Criterion	Evidence expected	Weight
Correct parallel forward	`v1` and `v2` bit-identical to the reference, including $T > B$ and an uneven partition	25 %
Correct parallel training	One local gradient per worker, fixed-order reduction, one update; gradient and parameter errors reported	25 %
Ownership and synchronization	Can point in the code to each owner, each shared read and each synchronization point, and justify where no lock is needed	20 %
Measurement protocol	Same scope for t_1 and t_T, at least five runs, median and spread, $B < T$ case	15 %
Interpretation	Amdahl with a measured φ; one limiting cost supported by a measurement	15 %

A Language-specific guidance

These patterns illustrate worker creation and joining. Each calls your `process_range` routine with a begin index, an end index, and a worker index; you must provide its computation, buffers, and ownership. Launch every worker before the join loop. Do not create nested teams accidentally when an outer image decomposition already uses all CPU workers.

A.1 C++17: capture a different worker index for each thread

Listing 11: A chunked thread-launch pattern in C++17.

```
#include <thread>
#include <vector>

// M >= 0 items; T >= 1 workers.
std::vector<std::thread> workers;
workers.reserve(T);
for (int tid = 0; tid < T; ++tid) {
    workers.emplace_back([&, tid] {
        const int begin = int((1LL * tid * M) / T);
        const int end   = int((1LL * (tid + 1) * M) / T);
        process_range(begin, end, tid);
    });
}
```

```
for (auto& worker : workers) {
    worker.join();
}
// All assigned work has completed here.
```

The capture `[&, tid]` copies `tid` into each worker while referencing the surrounding data. That surrounding data must remain alive until the workers finish. Preallocate output arrays and private workspaces before launching; avoid resizing shared containers during the calculation. Completion and joining supply the required synchronization before the joining thread consumes results [3].
Check. Start all workers before waiting for their completion, and make sure each worker receives a stable worker index.

A reusable barrier in C++17. `std::barrier` requires C++20. With C++17, the following class lets a persistent team separate its phases (call `arrive_and_wait()` at the end of each phase). The generation counter makes it reusable and protects against spurious wake-ups.

Listing 12: A minimal reusable barrier for C++17.

```
#include <condition_variable>
#include <mutex>

class Barrier {
    std::mutex m;
    std::condition_variable cv;
    const int n;
    int count;
    unsigned long generation = 0;
public:
    explicit Barrier(int n) : n(n), count(n) {}
    void arrive_and_wait() {
        std::unique_lock<std::mutex> lock(m);
        const unsigned long g = generation;
        if (--count == 0) {          // last arrival opens the barrier
            ++generation;
            count = n;
            cv.notify_all();
        } else {
            cv.wait(lock, [&] { return g != generation; });
        }
    }
};
```

The mutex also provides the required visibility: writes made before `arrive_and_wait()` are visible to every worker after it returns.

A.2 Java: one stable index and private scratch space per worker

Listing 13: Equivalent chunk ownership in Java.

```
Thread[] workers = new Thread[T];
for (int tid = 0; tid < T; ++tid) {
    final int id = tid;
    final int begin = (int) ((long) id * M / T);
    final int end = (int) ((long) (id + 1) * M / T);
    workers[id] = new Thread(() -> processRange(begin, end, id));
    workers[id].start();
}
for (Thread worker : workers) {
    worker.join();   // enclosing method handles InterruptedException
}
```

Each worker needs its own workspace and local gradient. Copying a two-dimensional array reference does not copy its rows. A fixed executor pool can later reuse workers. Java start/join rules provide visibility of completed results [4, 5].

A.3 Python: first identify the running interpreter

For CPU-bound loops written in Python, ordinary GIL-enabled CPython and free-threaded CPython have different parallel execution behaviour. Check the actual interpreter after importing the modules used in your program; a free-threaded build can run with the GIL enabled [6].

Listing 14: Inspect free-threading support and the active GIL.

```
import sys
import sysconfig

print(sys.version)
print("free-threaded build:",
      sysconfig.get_config_var("Py_GIL_DISABLED") == 1)
check_gil = getattr(sys, "_is_gil_enabled", None)
print("GIL enabled:", check_gil() if check_gil else "not exposed")
```

If your environment uses GIL-enabled CPython, record that limitation when interpreting CPU-bound thread timings, and follow the Python policy of Section 2. A library call may execute native code with a different threading policy, so record whether the convolution is implemented in your Python loops or delegated to a numerical library.

Listing 15: Launch all workers, then join all workers.

```
from threading import Thread

def worker(tid):
    begin = tid * M // T
    end = (tid + 1) * M // T
    process_range(begin, end, tid)

workers = [Thread(target=worker, args=(tid,)) for tid in range(T)]
for thread in workers:
    thread.start()
for thread in workers:
    thread.join()
```

`target` is the name of a constructor parameter, not a reserved Python keyword. `target=worker` passes the function; `target=worker(...)` would call the function while constructing the thread. The trailing comma in `(tid,)` makes a one-element tuple. Shared numerical updates still need a correct synchronization or ownership design; free-threading is not a substitute for that design.

B Debugging strategy

Observation	A possible cause	A useful next check
Wrong shape	Wrong stride, padding, or inclusive loop bound	Calculate the first and last valid window. For this model, the final convolution origin is $(25, 25)$.
Correct first filter only	Incorrect offset between channels or filters	Print the flat index of the first value of every feature map.
Correct with one worker, wrong with two	Shared accumulator, overlapping output, or a missing join	Mark the exact written addresses for two small chunks.

Observation	A possible cause	A useful next check
Some outputs never written	Incomplete partition or empty-chunk bug	Initialize outputs to a recognizable sentinel and verify full coverage after joining.
Different pooling results	Tie policy, incorrect window origin, or wrong stride	Use a hand-built window with tied maxima and compare argmax indices.
Gradient mismatch	Wrong transpose, lost sum, wrong ReLU mask, or repeated division by batch size	Compare one layer at a time and one mini-batch update from identical parameters.
Loss becomes nonfinite	Unstable exponentials, invalid access, or too large an update	Test stable softmax; inspect finite values before and after one update.
Training differs although no data race is detected	Different sample order, batches, or parameter snapshots	Freeze one batch and compare all gradients before any update.
Good accuracy but wrong intermediate values	Accuracy hides numerical differences	Compare activations, logits, gradients, and post-update parameters.
More workers run slower	Too little work, repeated launches, poor locality, or too many workers	Increase work per chunk at fixed overall workload and measure launch time separately.
Timing varies greatly	Too short a run, warm-up, system load, or inconsistent scope	Repeat sufficiently long runs, retain the raw durations, and report median and spread.
Kernel speeds up, full epoch barely changes	Other phases dominate or new overhead appears	Measure forward, backward, reduction, update, and runtime overhead separately.
Synthetic accuracy is very high	The artificial classification task is simple	Identify the dataset and verify real MNIST separately before making a digit-recognition claim.

Hint. Use the smallest test that can expose the suspected error. Start with a small matrix or a single image, fixed parameters, and two workers. Verify boundary outputs and an uneven division of work. Then increase the data size. Debug builds or a race detector can help investigate a specific failure; collect performance numbers with the optimized build used for both versions.

C Glossary for the laboratory

Term	Meaning in this lab
Tensor	A multidimensional array of numbers with a stated shape and axis order.
Channel	One input component or one feature map. A grayscale image has one input channel.
Filter / kernel	Learned coefficients applied over local input windows; a standard filter spans all input channels.
Feature map	Spatial array produced by one output channel of a convolutional layer.
Cross-correlation	The sliding dot product used here, without reversing filter indices. CNN software conventionally calls this a convolution.
Stride	Distance, in input positions, between consecutive window origins.
Padding	Values conceptually added outside the input boundary. The core convolution uses no padding.
Receptive field	Input region that can influence a specified output value.

Term	Meaning in this lab
Activation	Output of a neuron or layer; in this guide A specifically denotes the ReLU output.
ReLU	$\max(0, x)$; its derivative is 1 for positive inputs, 0 for negative inputs, and defined here as 0 at zero.
Max-pooling	Selection of the largest value in a local window; the winning index is needed for the backward pass.
Flatten	A mapping from multiple axes to one vector axis, preserving a specified ordering; it need not allocate a new copy.
Logit	An unnormalized class score before softmax.
Softmax	Normalization of logits into probabilities summing to one. Exact values are positive; floating-point underflow can produce zeros.
Loss	A scalar objective to minimize; here cross-entropy for the correct class.
Accuracy	Number of correct predicted labels divided by the number of evaluated images.
Gradient $\nabla_\theta L$	Vector of partial derivatives of loss with respect to the model parameters.
Backward pass	Chain-rule calculation of gradients through the stored forward intermediates.
Learning rate	Step size η; with this lab's summed gradients, $\theta \leftarrow \theta - (\eta/B)G$.
Mini-batch	Group of images whose gradients are combined before one update.
Epoch	One pass through the chosen training subset.
Inference	Forward computation with fixed model parameters; no parameter update.
Worker	A CPU thread or equivalent execution resource to which a chunk of work is assigned.
Private workspace	Scratch arrays owned by one worker, often reused for successive images.
Data race	In C++, potentially concurrent conflicting accesses, at least one non-atomic, with neither ordered before the other by happens-before. Such a race yields undefined behaviour.
Mutex / lock	A mutual-exclusion mechanism protecting a specified critical section.
Barrier	A synchronization point that a group must reach before proceeding. A join has a different API and waits for thread completion. `std::barrier` is C++20; Java has `CyclicBarrier`, Python `threading.Barrier`.
Reduction	Combining partial results with an operation such as addition. Floating-point grouping can affect the last digits.
Granularity	Amount of useful computation assigned to one task or chunk.
Locality	Reuse of nearby or recently accessed data in a way that can benefit the memory hierarchy.
False sharing	Performance interference when workers modify different variables that occupy the same cache line. It does not by itself mean they write the same variable.
Speedup	$S(T) = t_1/t_T$ for the same workload and stated timing scope.
Parallel efficiency	$E(T) = S(T)/T$.
Throughput	Work completed per unit time, for example images/s.
Strong scaling	Change in time as worker count increases while total workload stays fixed.
Weak scaling	Behaviour when workload grows with worker count under an explicitly defined growth rule.
Reproducibility	Ability to repeat the stated procedure and interpret its numerical and performance variability under a recorded configuration.

References and further reading

[1] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016, Chapters 6 and 9. https://www.deeplearningbook.org/.

[2] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. "Gradient-based learning applied to document recognition." *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. https://doi.org/10.1109/5.726791. MNIST dataset: https://yann.lecun.org/exdb/mnist/ (the original dataset page may be temporarily unavailable; `fetch_mnist.py` uses mirrors).

[3] ISO C++ Working Draft. *Concurrency support: Thread members and data races*. Sections [thread.thread.member] and [intro.races]. https://eel.is/c++draft/thread.thread.member; https://eel.is/c++draft/intro.races.

[4] Oracle. *Java SE 25 API Specification: Class Thread*. https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Thread.html.

[5] Oracle. *The Java Language Specification, Java SE 25 Edition*. Chapter 17: Threads and Locks. https://docs.oracle.com/javase/specs/jls/se25/html/jls-17.html.

[6] Python Software Foundation. *Python 3.14 Documentation: Python Support for Free Threading*. https://docs.python.org/3.14/howto/free-threading-python.html.

[7] G. M. Amdahl. "Validity of the single processor approach to achieving large scale computing capabilities." *Proceedings of the AFIPS Spring Joint Computer Conference*, pp. 483–485, 1967. https://doi.org/10.1145/1465482.1465560.